

Draft Thesis Plan

Implementation and Experimental Evaluation of Simplified Jordan Sorting Yixue Wang

Goal

The goal is to reconstruct, implement, verify, and experimentally evaluate the simplified Jordan-sorting framework of Fung, Nicholl, Tarjan, and Van Wyk, in relation to the earlier algorithm of Hoffmann, Mehlhorn, Rosenstiehl, and Tarjan.

Main Idea

I separate the algorithmic framework from the list representation used for sibling lists. I first implement the framework with ordinary(doubly linked lists) sibling lists as a reference backend. I do not claim that this implementation achieves the linear-time bound. Instead, I instrument the implementation to measure the cost of split operations and use this to explain why a more sophisticated finger-search / heterogeneous finger-tree representation is needed.

The doubly linked sibling-list implementation is intended as a correct, testable reference implementation, not as the full linear-time implementation.

Research Questions

1. How can the 1986 and 1990 Jordan-sorting algorithms be explained clearly with examples, invariants, and pseudocode?
2. How can the simplified framework be implemented with ordinary sibling lists while maintaining the sorted list and the upper/lower family trees?
3. How can correctness be checked automatically using standard sorting and laminarity checks for upper and lower pairs?
4. How does the ordinary-list backend behave experimentally, especially for split operations, split sizes, and traversal work?
5. Where exactly does the ordinary-list backend lose the linear-time bound, and how do level-linked search trees or heterogeneous finger trees address this bottleneck?

Planned Contributions

1. Clear explanation and comparison of the 1986 and 1990 Jordan-sorting papers.
2. Correct reference implementation of the simplified Jordan-sorting framework.
3. Correctness oracle based on ordinary sorting and laminarity checks.
4. Test generation for valid and invalid Jordan sequences.
5. Experimental comparison with merge sort and quick sorting.
6. Instrumented analysis of list-splitting costs and theoretical explanation of the linear-time list representations.
7. Theoretical explanation of how level-linked search trees and heterogeneous finger trees restore the linear-time bound.

Implementation Scope

- sorted list of processed elements
- upper and lower family trees
- sibling lists
- family-list splitting
- rejection of invalid inputs
- correctness oracle

Experimental Scope

- wall-clock time
- number of split operations
- size of split regions
- traversal work in ordinary sibling lists
- comparison with merge sort and quick sorting
- behavior on valid and invalid instances

Optional Extension

Prototype or discuss a more advanced backend, for example level-linked 2-4 trees, heterogeneous finger trees or splay trees.